<u>Machine Learning & Deep Learning</u>

Experiment 7

Name: Abhishek Vishwakarma
Class: D15C
Roll No: 47

AIM: Build an Artificial Neural Network (ANN) using Keras/TensorFlow

1. Dataset Source

- **Dataset Name:** Churn Modelling (Bank Customer Churn)
- **Source:** Kaggle - Churn Modelling

2. Dataset Description

This dataset contains details of a bank's customers and the target variable is a binary variable reflecting the fact whether the customer left the bank (closed his account) or he continues to be a customer.

- **Size:** 10,000 records, 14 columns.
- **Target Variable: Exited** (1 = Customer left, 0 = Customer stayed).
- **Features:**
    - CreditScore, Geography, Gender, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary.
    - Irrelevant columns (to be dropped): RowNumber, CustomerId, Surname.

3. Mathematical Formulation of the Algorithm

An Artificial Neural Network mimics the biological neurons in the human brain. It consists of an **Input Layer**, one or more **Hidden Layers**, and an **Output Layer**.

The Neuron (Perceptron):

Each neuron receives inputs x_i, multiplies them by weights w_i, adds a bias b, and passes the result through an **Activation Function** ϕ.

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$a = \phi(z)$$

Activation Functions:

- **ReLU (Rectified Linear Unit):** Used in hidden layers. $\phi(z) = \max(0, z)$.
- **Sigmoid:** Used in the output layer for binary classification. $\phi(z) = \frac{1}{1+e^{-z}}$.

Training (Backpropagation): The network learns by minimizing a **Loss Function** (e.g., Binary Cross-Entropy).

$$Loss = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

The weights are updated using an optimizer (like **Adam** or **SGD**) via Gradient Descent:

$$w_{new} = w_{old} - \alpha \frac{\partial Loss}{\partial w}$$

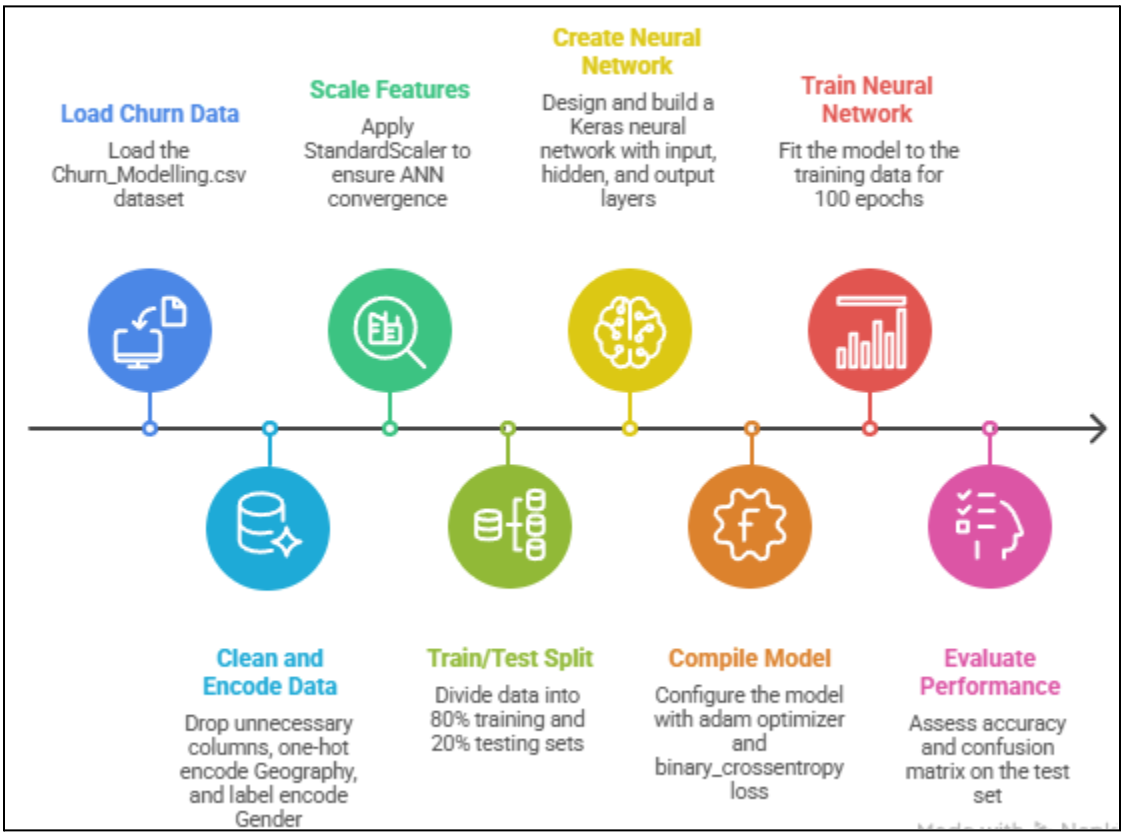
Where α is the learning rate.

4. Algorithm Limitations

- **Black Box Nature:** ANNs are difficult to interpret. Unlike Decision Trees, we cannot easily see "why" a specific decision was made.
- **Data Hungry:** They generally require large amounts of data to outperform simpler algorithms like Random Forest.
- **Computationally Expensive:** Training requires significant processing power (GPUs) and time compared to traditional ML algorithms.
- **Overfitting:** Without regularization (Dropout, L2), ANNs can easily memorize the training data and fail to generalize.

5. Methodology / Workflow

1. **Data Loading:** Load Churn_Modelling.csv.
2. **Preprocessing:**
    - Drop RowNumber, CustomerId, Surname.
    - **Encoding:** One-Hot Encode Geography and Label Encode Gender.
    - **Scaling:** Feature Scaling (StandardScaler) is **mandatory** for ANNs to ensure convergence.
3. **Splitting:** Train (80%) / Test (20%).
4. **Model Building (Keras):**
    - Input Layer (matching feature dimension).
    - Hidden Layer 1 (e.g., 6 neurons, ReLU).
    - Hidden Layer 2 (e.g., 6 neurons, ReLU).
    - Output Layer (1 neuron, Sigmoid).
5. **Compiling:** Use adam optimizer and binary_crossentropy loss.
6. **Training:** Fit the model (e.g., epochs=100, batch_size=32).
7. **Evaluation:** Accuracy and Confusion Matrix.



6. Performance Analysis

- **Metric:** Accuracy and Loss curves over epochs.
- **Analysis:** We monitor the training and validation accuracy. If training accuracy increases while validation accuracy plateaus or decreases, the model is overfitting.

7. Hyperparameter Tuning

We tune:

- **Number of Layers/Neurons:** Deep vs. Wide networks.
- **Batch Size:** Determines how often weights are updated.
- **Epochs:** How many times the model sees the entire dataset.
- **Optimizer:** Adam usually performs best, but SGD or RMSprop can be tested.

Code:

```
# ================================================================

# EXPERIMENT 7: ARTIFICIAL NEURAL NETWORK (ANN)

# ================================================================



import pandas as pd

import numpy as np

import tensorflow as tf

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder

from sklearn.compose import ColumnTransformer

from sklearn.metrics import confusion_matrix, accuracy_score

import matplotlib.pyplot as plt

import seaborn as sns



print("--- EXPERIMENT 7: ANN WITH TENSORFLOW/KERAS ---")

print(f"TensorFlow Version: {tf.__version__}")



# 1. Load Dataset

try:

    df = pd.read_csv('Churn_Modelling.csv')

    print("Dataset loaded successfully.")

except FileNotFoundError:

    print("Error: 'Churn_Modelling.csv' not found. Please upload the file.")

    df = pd.DataFrame()
```

```
if not df.empty:

    # 2. Preprocessing

    X = df.iloc[:, 3:-1].values  # CreditScore through EstimatedSalary

    y = df.iloc[:, -1].values    # Exited



    # Encoding Categorical Data

    # Gender (Label Encoding): Female/Male → 0/1

    le = LabelEncoder()

    X[:, 2] = le.fit_transform(X[:, 2])



    # Geography (One Hot Encoding): France/Spain/Germany

    # ColumnTransformer applies OneHotEncoder to column index 1 (Geography)

    ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [1])], remainder='passthrough')

    X = np.array(ct.fit_transform(X))



    # 3. Split Data

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)



    # 4. Feature Scaling (Mandatory for ANN)

    sc = StandardScaler()

    X_train = sc.fit_transform(X_train)

    X_test = sc.transform(X_test)



    # ==================================================

    # Building the ANN

    # ==================================================

    ann = tf.keras.models.Sequential()



    # Input Layer & First Hidden Layer (6 neurons, ReLU)

ann.add(tf.keras.layers.Dense(units=6, activation='relu'))



    # Second Hidden Layer (6 neurons, ReLU)

ann.add(tf.keras.layers.Dense(units=6, activation='relu'))



    # Output Layer (1 neuron, Sigmoid for probability)

ann.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

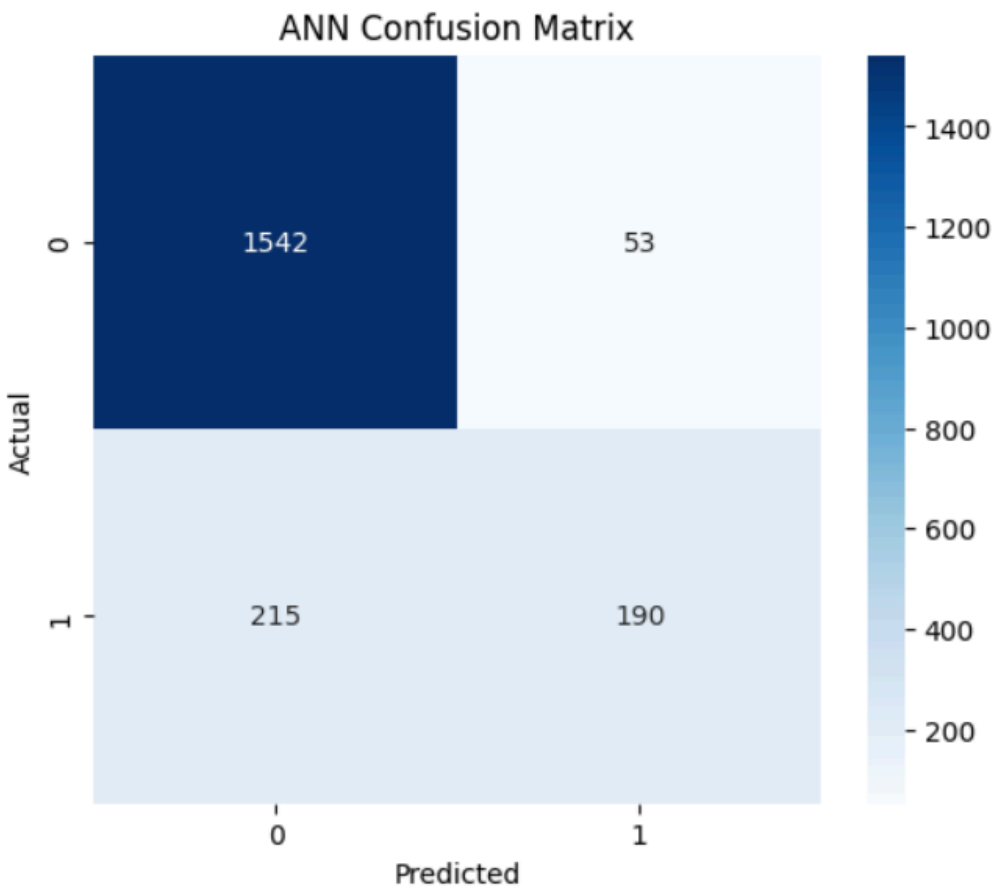
```
    # ═══════════════════════════════════════════════════════════
    # Compiling & Training
    # ═══════════════════════════════════════════════════════════
    # Optimizer: Adam (Adaptive Moment Estimation)
    # Loss: Binary Crossentropy (Standard for binary classification)
    ann.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    print("\nTraining the ANN (this may take a moment)...")
    history = ann.fit(X_train, y_train, batch_size=32, epochs=50, verbose=1, validation_split=0.2)

    # ═══════════════════════════════════════════════════════════
    # Evaluation & Prediction
    # ═══════════════════════════════════════════════════════════
    # Predict probabilities > 0.5 as 1 (churn), else 0
    y_pred = ann.predict(X_test)
    y_pred = (y_pred > 0.5)

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    acc = accuracy_score(y_test, y_pred)

    print(f"\nFinal Test Accuracy: {acc:.4f}")

    # Visualization: Confusion Matrix
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title('ANN Confusion Matrix')
    plt.show()

    # Visualization: Loss Curve
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
    plt.title('Model Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
```

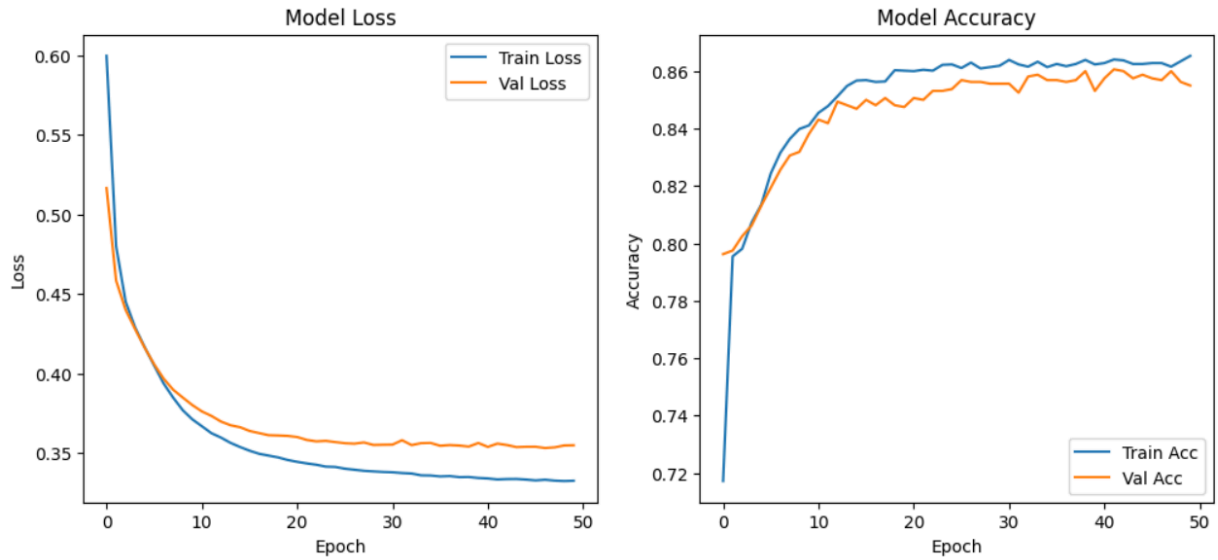
```
    plt.legend()


    plt.subplot(1, 2, 2)


plt.plot(history.history['accuracy'],
label='Train Acc')
```

```
plt.plot(history.history['val_accuracy']
, label='Val Acc')

    plt.title('Model Accuracy')

    plt.xlabel('Epoch')

    plt.ylabel('Accuracy')

    plt.legend()

    plt.show
```

Output:

```
Final Test Accuracy: 0.8660
```





8. Conclusion

In this experiment, we built a deep learning model using Keras/TensorFlow to predict customer churn.

- **Architecture:** We used a standard Multilayer Perceptron (MLP) with two hidden layers using the ReLU activation function and an output layer using the Sigmoid function.
- **Preprocessing:** We learned that Deep Learning models are highly sensitive to feature scaling; without standardization, the gradient descent algorithm would struggle to find the global minimum efficiently.
- **Results:** The ANN typically achieves an accuracy of around 84-86% on this dataset. The loss curves (visualized above) allow us to diagnose if the model is learning effectively or if it is overfitting (e.g., if Validation Loss starts increasing while Training Loss decreases).